PODRĘCZNIK OPERATORA

Bot telefoniczny AI. Uruchom i podłącz firmę.

Twilio + OpenAI + Telegram.
Projekt bazowy, komendy, konfiguracja i testy.
Pierwsza firma i kolejne wdrożenie.

01	02	03	04	05
Telefon	Twilio	Bot + AI	Zgłoszenie	Telegram

2 rozdziałów • Projekt kodu w panelu DAWKOZ • lokalne demo bez płatnych API
Wersja 1.1 | 26.10.2026 | Oferta DAWKOZ 1.1

Instrukcja do projektu bazowego. Próby lokalne nie potwierdzają rozmowy z operatorem ani naliczenia kosztów. Etap live wymaga własnych kont, środków, numeru, ustawień i testów. Starter nie zawiera SMS ani centrali IVR.

Spis treści

Kliknij tytuł rozdziału lub skorzystaj z zakładek PDF. Zacznij od lokalnego demo projektu, następnie ustaw konta i przejdź testy telefonu.

1. Technicznie: uruchom i spersonalizuj bota bazowego 3

2. Kolejna firma — osobny bot od briefu do odbioru 19

1. Technicznie: uruchom i spersonalizuj bota bazowego

Konkretna ścieżka od komputera i kodu do testowego numeru, rozmowy AI i zgłoszenia w Telegramie. Polecenia i konfiguracja odnoszą się do dołączonego startera DAWKOZ.

To jest instrukcja techniczna wybranego wariantu
Ten rozdział dotyczy konkretnego startera DAWKOZ: jedna firma, osobna instancja, osobna baza SQLite, numer Twilio, rozpoznawanie mowy w Twilio, odpowiedzi głosowe Twilio oraz model OpenAI do uporządkowania odpowiedzi. Zgłoszenie trafia do jednego prywatnego odbiorcy w Telegramie. Rozmowa przebiega turami: bot pyta, rozmówca odpowiada, bot przetwarza odpowiedź. To nie jest swobodna rozmowa Realtime z nieograniczonym przerywaniem.

Nie sprzedawaj samego startera jako całego pakietu
Starter jest bazą do uruchomienia i sprawdzenia wybranego procesu. Nie zawiera SMS, IVR, przekierowania rozmowy na telefon pracownika, rezerwacji terminu, KSeF ani kont klienta w panelu. W ofercie bota jest również limit SMS; przed realizacją pełnego pakietu trzeba wdrożyć i sprawdzić brakujące elementy lub pisemnie uzgodnić inny zakres. W tej sesji nie zakupiono numeru i nie wykonano prawdziwego płatnego połączenia u dostawców.

Pobierz projekt bota — ZIP (po zalogowaniu)
https://dawkoz.com/app/guide/bot-starter.zip

Pobierz osobną instrukcję techniczną PDF (po zalogowaniu)
https://dawkoz.com/app/guide/bot-technical.pdf

Pracuj najpierw na firmie demonstracyjnej i danych fikcyjnych. Dopiero po poprawnym teście lokalnym zakładaj oddzielne konto/projekt do pierwszej firmy. Kolejność jest celowa: komputer → test bez API → konfiguracja firmy → konta → hosting → połączenie testowe → odbiór → rzeczywisty numer. W panelu administracyjnym DAWKOZ przygotowujesz dokumenty i czytasz instrukcję; nie uruchamiasz tam automatycznie usługi telefonu.

Element	Po co jest potrzebny	Co robi Dawid
Komputer z Node.js 24	Uruchomienie dostarczonego kodu i lokalnej bazy SQLite.	Instaluje oficjalny Node.js 24 LTS lub zgodną wersję 24.x i sprawdza node --version. Nie używa nieznanego instalatora.
Projekt bota	Kod, konfiguracja przykładowej firmy, polecenia testowe i Dockerfile.	Pobiera archiwum, rozpakowuje do oddzielnego folderu, wykonuje opisane testy.
Twilio	Numer, połączenie telefoniczne, rozpoznawanie mowy i synteza głosu.	Tworzy lub otrzymuje dostęp do właściwego projektu/konta, weryfikuje kraj i numer oraz ustawia adresy webhooków.

Element	Po co jest potrzebny	Co robi Dawid
OpenAI Platform	Przetworzenie wypowiedzi do ograniczonej struktury.	Tworzy projekt API i klucz; osobno konfiguruje rozliczenia API. Abonament ChatGPT Plus nie finansuje API.
Telegram	Przekazanie gotowej sprawy do ustalonego odbiorcy.	Tworzy oddzielnego bota i prywatną grupę/czat, sprawdza odbiorcę przed użyciem prawdziwych danych.
Railway	Publiczny adres HTTPS, uruchomienie Node.js i trwały dysk bazy.	Tworzy oddzielną usługę od strony dawkoz.com, ustawia zmienne, jeden proces i volume. Hosting i volume mogą być płatne.

A. Przygotuj komputer i folder projektu

Pierwsze uruchomienie bez wydatków u dostawców

1. Przygotuj komputer, nie tylko telefon

Na telefonie możesz czytać instrukcję i panel, ale pierwsze wdrożenie wykonuj na komputerze. Otwórz stronę nodejs.org i pobierz oficjalny instalator wersji 24.19 lub nowszej 24.x odpowiedniej dla systemu. Po instalacji zamknij i otwórz ponownie Terminal / PowerShell. Nie instaluj kilku różnych wersji przypadkowo.

Sprawdź wynik: Polecenie node --version zwraca v24.19 lub nowsze 24.x; jeżeli system nie znajduje polecenia, najpierw napraw instalację.

2. Pobierz i rozpakuj projekt

Po zalogowaniu do panelu DAWKOZ pobierz „Projekt bota — ZIP” z instrukcji technicznej. W Windows kliknij ZIP prawym przyciskiem i wybierz „Wyodrębnij wszystkie”. Rozpakuj do nowego folderu, np. Dokumenty/dawkoz-bot-demo, i otwórz zawarty w nim podfolder bot-starter. Nie uruchamiaj kodu wewnątrz archiwum. Pobierane archiwum nie zawiera Twoich kluczy ani bazy rzeczywistej firmy.

Sprawdź wynik: W zwykłym folderze widzisz package.json, README i plik przykładowej konfiguracji; nie zagnieżdżony ZIP.

3. Otwórz terminal we właściwym folderze

Otwórz rozpakowany folder w Eksploratorze, kliknij prawym przyciskiem w pustym miejscu i wybierz „Otwórz w terminalu”; w starszym Windows użyj paska adresu folderu i wpisz powershell. W macOS otwórz Terminal i przejdź poleceniem cd do rozpakowanego folderu. Polecenia z tego rozdziału wykonujesz w folderze bota, a nie w repozytorium dawkoz-website.

Sprawdź wynik: Polecenie dir w Windows albo ls w macOS pokazuje package.json tego startera.

4. Sprawdź dołączone testy

Uruchom npm test. Kod nie wymaga instalowania bibliotek npm do działania: korzysta z Node.js 24.19 lub nowszego 24.x i jego modułów wbudowanych. Testy mają podstawione odpowiedzi dostawców; nie wymagają klucza ani zakupu numeru. Ostrzeżenie Node o eksperymentalnym module SQLite samo nie oznacza nieudanego testu; sprawdź końcowy wynik. Nie pomijaj błędu walidacji lub testu.

Sprawdź wynik: Testy kończą się poprawnie; brak testów na żywym koncie nie jest potwierdzeniem kosztu i jakości prawdziwej rozmowy.

5. Utwórz własny plik ustawień

W nowym folderze uruchom npm run init. Powstaną lokalny .env oraz config/client.json z bezpiecznych przykładów, bez nadpisania istniejących plików. Otwórz je w zwykłym edytorze tekstu, np. VS Code; zachowaj nazwy i UTF-8. W .env pozostaw BOT_MODE=demo i ALLOW_LIVE_CALLS=false. Nie wpisuj jeszcze kluczy. CLIENT_CONFIG_JSON pozostaw nieustawione lokalnie, bo nadpisuje treść pliku konfiguracyjnego.

Sprawdź wynik: Plik lokalny istnieje pod dokładną nazwą; Windows nie dopisał ukrytego rozszerzenia .txt.

6. Uruchom lokalny demonstrator

Najpierw wykonaj npm run check, a następnie npm run demo — to automatyczna próba offline bez połączeń z dostawcami. Aby zobaczyć ekran, uruchom npm start i otwórz http://localhost:3001/demo. Wpisuj wyłącznie dane fikcyjne. Aplikacja demonstruje kolejne etapy tekstowo; nie jest mikrofonem ani numerem telefonu i nie potwierdza jakości mowy OpenAI/Twilio.

Sprawdź wynik: Dane przykładowe przechodzą przez kolejne pola, a brak kluczy w trybie demo nie uruchamia zewnętrznych API.

7. Zatrzymaj demonstrator przed zmianą konfiguracji

W terminalu naciśnij Ctrl+C. Konfiguracja ładuje się przy starcie. Przed zmianą firmy z przykładowej na rzeczywistą ustaw osobny DATA_DIR, np. ./data-firma-b-test, i unikalny clientId; stara baza demonstracyjna pozostaje oddzielona. Istniejąca baza wiąże się z clientId i companyName, więc zmiana właściciela lub samej nazwy wymaga nowego dysku albo świadomej migracji. Nie usuwaj bazy działającego klienta.

Sprawdź wynik: Nowy katalog testowy należy do jednej firmy; restart nie podmienił istniejącej bazy.

Polecenia: w folderze bot-starter

```
node --version
npm run init
npm test
npm run check
npm run demo
npm start
```

Wykonuj kolejno. Ostatnie polecenie zostawia serwer uruchomiony; wtedy przejdź do przeglądarki. Po Ctrl+C terminal wraca do wpisywania poleceń. Przy komunikacie EADDRINUSE zatrzymaj inną lokalną usługę lub świadomie wybierz inny PORT i odpowiedni adres. Nie uruchamiaj kilku procesów z tą samą bazą. Npm run init nie nadpisuje Twoich plików: jeśli chcesz nowe demo, użyj nowego folderu, nie kasuj bazy aktywnego klienta.

```
http://localhost:3001/demo
http://localhost:3001/healthz
```



Rzeczywisty demonstrator startera w trybie offline. To tekstowa próba procesu na danych przykładowych, bez połączenia telefonicznego, bez mikrofonu i bez kosztów API.

B. Spersonalizuj jedną firmę, zanim podłączysz numer

Materiały i konfiguracja klienta

1. Opisz zadanie bota w jednym zdaniu

Ustal z firmą jeden proces, np. „Przyjąć usterkę, zebrać oznaczenie obiektu, opis problemu i kontakt oraz przekazać sprawę ekipie". Nie dodawaj do tego samodzielnie umawiania wizyt, sprawdzania zaległości i wyceny prac. Wypisz też czynności, których bot ma nie wykonywać.

Sprawdź wynik: Klient zatwierdził jeden wynik i rozumie granice działania.

2. Przygotuj krótkie powitanie

Wpisz rzeczywistą nazwę firmy, informację o AI oraz cel rozmowy. Przykład: „Dzień dobry, tu automatyczny asystent AI firmy DEMO. Przyjmę zgłoszenie usterki. Proszę podać oznaczenie obiektu". Nie mów, że po drugiej stronie jest człowiek. Nagrywanie nie jest włączone samym powitaniem; wymaga odrębnych ustaleń.

Sprawdź wynik: Treść jest krótka, zrozumiała i zaakceptowana przez klienta; nie obiecuje przyjazdu ani terminu naprawy.

3. Ustal trzy zbierane pola

Dla pierwszego wariantu wybierz trzy pola pasujące do procesu. Każde powinno mieć prostą nazwę, pytanie, wymagany format i ograniczenie długości. Dane do kontaktu wpisuje rozmówca; numer dzwoniącego sam nie potwierdza jego tożsamości ani dostępu do danych mieszkańców.

Sprawdź wynik: Wiesz, jakie trzy odpowiedzi mają znaleźć się w zgłoszeniu; nie zbierasz PESEL, danych zdrowotnych ani innych zbędnych informacji.

4. Dodaj zatwierdzone FAQ

Poproś klienta o nie więcej niż 20 pytań i odpowiedzi związanych z tym procesem. Zapisz je zgodnie ze schematem pliku konfiguracyjnego. Podaj rzeczywiste godziny, zasady kontaktu i sposób obsługi braku odpowiedzi. Nie wstawiaj promocji, cen i terminów, których firma nie zatwierdziła.

Sprawdź wynik: Na pytanie testowe bot używa zatwierdzonej treści, a przy nieznanym temacie nie wymyśla danych.

5. Ustal zakończenie i oddanie sprawy człowiekowi

Klient zatwierdza zakończenie po potwierdzeniu danych i komunikat przy prośbie o pracownika. W tym starterze prośba o człowieka kończy rozmowę z informacją o kontakcie bezpośrednim, bez utworzenia zgłoszenia i bez przekierowania. Jeśli firma wymaga oddzwonienia lub połączenia z pracownikiem, to dodatkowa logika do wykonania i testu. Rozróżniaj „zgłoszenie zapisane do przekazania” od „wiadomość doręczona”.

Sprawdź wynik: W scenariuszu nie ma niewdrożonej obietnicy przełączenia, rezerwacji lub potwierdzenia wykonania zlecenia.

6. Sprawdź plik i próbę lokalną

Zapisz config/client.json w UTF-8. Uruchom npm run check, następnie npm run demo, a do sprawdzenia ekranu npm start. Cudzysłowy JSON muszą być zwykłe, bez końcowego przecinka. Jeśli zmiany w pliku nie działają, sprawdź, czy CLIENT_CONFIG_JSON nie jest ustawione: jeśli jest, ma pierwszeństwo. Na hostingu konfigurację tej firmy podasz właśnie przez tę zmienną. Npm run demo korzysta z przykładu dołączonego do kodu; własne zmienione pytania sprawdzaj ręcznie na ekranie /demo.

Sprawdź wynik: Konfiguracja przechodzi kontrolę; nazwa firmy, trzy pytania, FAQ i podsumowanie odpowiadają ustaleniom.

Przykładowa zawartość config/client.json

```
{
"clientId": "firma-demo",
"companyName": "Firma DEMO",
"greeting": "Dzień dobry. Jestem asystentką AI firmy DEMO. Przyjmę zgłoszenie dla
zespołu. Proszę nie podawać danych wrażliwych.",
"processDescription": "Przyjmowanie zgłoszeń usterek. Bez obietnicy rezerwacji,
ceny ani terminu naprawy.",
"language": "pl-PL",
"voice": "Polly.Ewa",
"fields": [
{
"key": "address",
"label": "Adres obiektu",
"question": "Podaj adres obiektu, którego dotyczy zgłoszenie.",
"required": true,
"maxLength": 240
},
{
"key": "issue",
"label": "Opis usterki",
"question": "Opisz krótko usterkę.",
"required": true,
"maxLength": 500
},
{
"key": "contact",
"label": "Kontakt do zgłaszającego",
"question": "Podaj numer telefonu lub inny kontakt dla pracownika.",
"required": true,
"maxLength": 160
}
],
"faq": [
{
"question": "Czy zgłoszenie oznacza rezerwację naprawy?",
"answer": "Nie. Zespół sprawdzi zgłoszenie i skontaktuje się, aby ustalić
dalsze działania."
},
{
"question": "Czy rozmawiam ze sztuczną inteligencją?",
"answer": "Tak, jestem asystentką AI i zbieram informacje dla pracownika."
}
],
"monthlyMinutes": 200,
"maxCallSeconds": 300,
"maxTurns": 12,
"maxConcurrentCalls": 1,
"contentRetentionDays": 7
}
```

To wyłącznie firma przykładowa. Zmień nazwę, powitanie, opis procesu, etykiety, pytania i zatwierdzone FAQ; w pierwszej konfiguracji zachowaj klucze address, issue i contact. Nie dodawaj sekretów do JSON. Voice Polly.Ewa jest ustawieniem do weryfikacji na Twoim koncie Twilio, nie potwierdzoną w tej sesji dostępnością. MaxTurns=12 ogranicza liczbę tur, a maxConcurrentCalls=1 to zamierzona prostota pierwszego wariantu. Miesięczne minuty muszą odpowiadać podpisanej umowie; nie zwiększaj ich po cichu w celu obejścia odrzucenia. clientId identyfikuje firmę i wraz z companyName wiąże bazę z jej właścicielem. Zmiana

właściciela wymaga nowej instancji i pustego, osobnego dysku. fields ma od 1 do 6 pól; w tym wariancie stosuj required=true. contentRetentionDays=7 określa dni przechowywania treści w SQLite.

C. Przygotuj konta dostawców

Przyciski w zewnętrznych panelach
Ścieżki poniżej opisują typowe sekcje kont Twilio, OpenAI i Railway. Dostawcy mogą zmienić układ lub nazwy. Gdy nie widzisz podanej sekcji, użyj wyszukiwarki w ich panelu lub dokumentacji. Nie zaznaczaj przypadkowej opcji płatnej i nie przesyłaj do ChatGPT zrzutu z widocznym tokenem.

Twilio — konto, numer i dane do integracji

1. Wejdź do właściwego projektu Twilio

Otwórz console.twilio.com, zaloguj się i sprawdź wybrany projekt/konto. Włącz 2FA. Dla firmy użyj uzgodnionego właściciela i uprawnień; nie mieszaj kilku klientów w jednym folderze bazy i nie dziel ich tokenu z nieuprawnioną osobą. Przed zakupem przeczytaj wymagania i cenę numeru.

Sprawdź wynik: Wiesz, kto jest właścicielem konta i kto opłaca numer, połączenia, mowę oraz pozostałe usługi.

2. Sprawdź numer do prób

W sekcji Phone Numbers → Manage → Active numbers sprawdź istniejący numer z obsługą Voice. Jeśli nie masz numeru, odszukaj zakup numeru i sprawdź dostępność kraju, wymagania adresowe/rejestracyjne i opłaty przed potwierdzeniem zakupu. Nie zakładaj, że polski numer jest zawsze dostępny od ręki.

Sprawdź wynik: Masz legalnie przypisany numer w formacie E.164, np. +48..., z obsługą przychodzących połączeń Voice.

3. Uwzględnij ograniczenia konta próbnego

Na trial mogą obowiązywać weryfikacje numerów, ograniczenia krajów i komunikaty próbne. Przetestuj własny numer i sprawdź ograniczenia widoczne w swoim koncie. Nie kieruj ruchu rzeczywistych klientów na trial jako na odebraną usługę produkcyjną.

Sprawdź wynik: Potrafisz wykonać dozwoloną próbę i rozumiesz, które ograniczenia znikną dopiero po weryfikacji lub zmianie konta.

4. Przenieś wymagane dane do bezpiecznej konfiguracji

Na stronie informacji konta znajdź Account SID i Auth Token. Token pokaż tylko na własnym ekranie i wstaw bezpiecznie do zmiennych środowiskowych usługi. Nie myl Auth Token używanego do weryfikacji podpisów Twilio z nazwą projektu albo sekretami innego konta. Nie zapisuj tych wartości w client.json, GitHubie ani instrukcji dla klienta.

Sprawdź wynik: SID i token pochodzą z tego samego właściwego konta; w repozytorium i wiadomościach nie ma sekretu.

5. Nie podłączaj jeszcze ruchu klienta

Na tym etapie zachowaj obecny numer i obsługę firmy bez zmian. Adresy webhooków wpiszesz dopiero po uruchomieniu oddzielnej usługi HTTPS i sprawdzeniu zmiennych. Zapisz dotychczasowe ustawienia numeru, żeby dało się do nich wrócić.

Sprawdź wynik: Masz plan cofnięcia i nie zastąpiłeś działającego telefonu pustym adresem testowym.

OpenAI Platform — projekt API

1. Otwórz platform.openai.com

Zaloguj się do platformy deweloperskiej OpenAI i wybierz organizację oraz projekt przeznaczony do tej firmy lub izolowanego środowiska. Rozliczenie API jest odrębne od używania ChatGPT na stronie lub w aplikacji. Sam ChatGPT Plus nie daje automatycznie środków na połączenia API.

Sprawdź wynik: Wiesz, z jakiego projektu będzie pochodził ruch i na czyim rachunku pojawi się zużycie.

2. Ustaw rozliczenia i budżet projektu

W ustawieniach rozliczeń sprawdź wymagane środki i metodę płatności. Ustaw alerty i ograniczenia dostępne na Twoim koncie. Traktuj próg budżetowy jako funkcję do sprawdzenia, nie pewną twardą blokadę: część limitów kosztowych tylko powiadamia. Limity aplikacji nadal muszą działać.

Sprawdź wynik: Masz mały, świadomie uzgodniony budżet testowy i wiesz, co nastąpi po jego przekroczeniu.

3. Utwórz właściwy klucz API

W API keys utwórz nowy klucz dla danego projektu. Jeżeli są dostępne ograniczenia uprawnień, dobierz minimum potrzebne do używanego endpointu. Zapisz klucz od razu w menedżerze haseł i zmiennych środowiskowych. Nie wrzucaj go do repozytorium, pliku konfiguracji firmy, PDF ani czatu.

Sprawdź wynik: Aplikacja zna klucz przez zmienną, a w kodzie widnieje tylko nazwa tej zmiennej.

4. Sprawdź model i sposób zapisu danych

Wybierz model obsługujący używany przez starter endpoint i ograniczoną strukturę odpowiedzi. Najpierw wykonaj mały test fikcyjnym zdaniem. Program ma używać store:false; to nie jest samodzielna gwarancja zerowej retencji całej usługi. Rzeczywiste warunki konta i przetwarzania potwierdź osobno.

Sprawdź wynik: API odpowiada na kontrolowane zapytanie; błąd uprawnień/modelu nie powoduje przejścia do niekontrolowanego innego modelu.

Telegram — oddzielny bot i właściwy odbiorca

1. Utwórz bota u oficjalnego BotFather

W Telegramie otwórz oficjalne, zweryfikowane konto @BotFather. Wyślij /newbot, podaj nazwę widoczną oraz wolną nazwę użytkownika kończącą się bot. Zapisz otrzymany token w bezpiecznej konfiguracji. Nie używaj przypadkowego konta podszywającego się pod BotFather.

Sprawdź wynik: Masz bota utworzonego dla ustalonej firmy i token, który nie trafił do publicznego pliku.

2. Wybierz prywatny czat z uprawnioną osobą

W pierwszym wdrożeniu użyj prywatnego czatu jednej osoby wskazanej przez firmę. Z jej konta otwórz bota i naciśnij Start lub wyślij /start. Ten dołączony pomocnik odczytuje wyłącznie ID prywatnych czatów; wdrożenie grupy wymaga osobnego sprawdzenia uprawnień i ID.

Sprawdź wynik: Firma wskazała właściwą osobę i zatwierdziła zakres przekazywanych danych.

3. Odczytaj identyfikator odbiorcy dołączonym narzędziem

W lokalnym .env wpisz TELEGRAM_BOT_TOKEN i uruchom npm run telegram:chat-id. Narzędzie sprawdza getWebhookInfo i getUpdates, bez drukowania tokenu ani adresu z tokenem. Najpierw wyślij /start z konta docelowego odbiorcy. Wpisz właściwy prywatny chat ID do TELEGRAM_CHAT_ID. Jeśli narzędzie pokaże kilka ID, nie zgaduj: potwierdź odbiorcę na jego koncie i wykonaj fikcyjne zgłoszenie. W razie wątpliwości utwórz nowego bota i rozpocznij czat tylko z ustaloną osobą.

Sprawdź wynik: Próbna wiadomość trafia do wskazanej osoby; nie zawiera danych rzeczywistego klienta.

4. Nie usuwaj obcego webhooka

Jeśli odczyt aktualizacji zwraca konflikt z istniejącym webhookiem, sprawdź, czy nie używasz bota działającego już w innym systemie. Dla nowej firmy najlepiej utworzyć oddzielnego bota. Nie usuwaj istniejącej integracji tylko po to, żeby uruchomić test.

Sprawdź wynik: Żaden dotychczasowy system nie przestał działać wskutek przygotowania nowego wdrożenia.

5. Wyślij wyłącznie fikcyjne zgłoszenie

Sprawdź dostarczenie komunikatu do uzgodnionego odbiorcy. Porównaj identyfikator zgłoszenia, trzy pola i podpis firmy. Jeśli wysyłka zakończyła się timeoutem, sprawdź czat przed ponowieniem: Telegram może przyjąć wiadomość, mimo że program nie dostał odpowiedzi. Nie obiecuj dokładnie jednego doręczenia przy niejednoznacznym błędzie sieci.

Sprawdź wynik: Zespół widzi jedną identyfikowalną sprawę, a status aplikacji rozróżnia dostarczenie, oczekiwanie i niepewność.

D. Wdróż oddzielną usługę na Railway

Zmienna	Lokalne demo	Próba live / Railway
BOT_MODE	demo	live dopiero po przygotowaniu kont.
ALLOW_LIVE_CALLS	false	true tylko dla świadomie rozpoczętego testu.
LIVE_TEST_ONLY	true	Pozostaje true w próbach. ALLOWED_CALLERS ogranicza numery dzwoniących.
ALLOWED_CALLERS	Niepotrzebne do offline	Twój zweryfikowany numer próbny w E.164; nie dowolny publiczny ruch.
CLIENT_CONFIG	config/client.json	Może pozostać przykładem; właściwa konfiguracja klienta przez CLIENT_CONFIG_JSON.
CLIENT_CONFIG_JSON	Zwykle nie ustawiaj	Cała zawartość zatwierdzonego JSON jako wartość zmiennej; ma pierwszeństwo przed plikiem.
DATA_DIR	./data	/data, zgodnie z volume. Jedna instancja i jedna baza na firmę.
PUBLIC_BASE_URL	Nie jest numerem telefonu	Dokładny publiczny adres HTTPS Railway bez końcowego / i bez ścieżki.
TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN	Puste	Wartości właściwego konta; token traktuj jak hasło.
TWILIO_PHONE_NUMBER	Puste	Numer Twilio z Voice, w formacie +48… lub innym rzeczywiście posiadanym.
OPENAI_API_KEY / OPENAI_MODEL	Puste / model przykładowy	Klucz projektu; model startowy gpt-4.1-mini wymaga dostępu na koncie.

Zmienna	Lokalne demo	Próba live / Railway
TELEGRAM_BOT_TOKEN / TELEGRAM_CHAT_ID	Puste	Token oddzielnego bota i dokładny prywatny odbiorca.
BILLING_PERIOD_START / BILLING_PERIOD_END	Według przykładu inicjalizacji	Jawne bieżące daty ISO z Z lub offsetem; po końcu okresu aplikacja blokuje nowe rozmowy.
PORT	Domyślnie 3001	Użyj wartości przekazanej przez Railway, nie wymuszaj jej bez potrzeby.

```
# Początek lokalnego .env — bez kluczy
BOT_MODE=demo
ALLOW_LIVE_CALLS=false
LIVE_TEST_ONLY=true
CLIENT_CONFIG=config/client.json
DATA_DIR=./data
```

```
# Ustawienia próby live — zamień WSZYSTKIE przykłady
BOT_MODE=live
ALLOW_LIVE_CALLS=true
LIVE_TEST_ONLY=true
ALLOWED_CALLERS=+48TWOJ_NUMER_PROBNY
PUBLIC_BASE_URL=https://TWOJ_DOKLADNY_HOST.up.railway.app
DATA_DIR=/data
TWILIO_ACCOUNT_SID=AC_TWOJ_SID
TWILIO_AUTH_TOKEN=TWÓJ_SEKRET_Z_KONTA
TWILIO_PHONE_NUMBER=+48POSIADANY_NUMER_TWILIO
OPENAI_API_KEY=TWÓJ_KLUCZ_API
OPENAI_MODEL=gpt-4.1-mini
TELEGRAM_BOT_TOKEN=TWÓJ_TOKEN_BOTA
TELEGRAM_CHAT_ID=TWÓJ_PRYWATNY_CHAT_ID
BILLING_PERIOD_START=2026-10-25T08:00:00Z
BILLING_PERIOD_END=2026-11-25T08:00:00Z
```

Przykłady nie są Twoimi danymi
Blok live jest ściągą nazw, nie gotowym zestawem do ślepego uruchomienia. Wszystkie wielkie opisy zastąp prawdziwymi wartościami w bezpiecznych zmiennych konta. Daty są przykładem z dnia wydania instrukcji: wpisz faktyczny bieżący okres klienta. W listopadzie 08:00Z oznacza 09:00 czasu polskiego; nie zakładaj stałej różnicy przez cały rok. Nie ustawiaj przyszłego/poprzedniego okresu, żeby ominąć naliczanie.

Hosting, sekrety i trwała baza

1. Przygotuj osobne prywatne repozytorium bota

Do wysłania kodu rozpakuj jeszcze raz świeży ZIP do oddzielnego folderu, np. dawkoz-bot-kod, i w tej kopii NIE wykonuj npm run init. Na GitHubie wybierz New repository, ustaw Private i utwórz repozytorium. Przez Add file → Upload files prześlij zawartość świeżego bot-starter, nie pliki z folderu, w którym prowadziłeś testy. Sprawdź listę przed Commit changes. .gitignore nie jest blokadą ręcznego wgrywania plików w przeglądarce: .env, config/client.json i data/ nie mogą trafić do repo. W repo zostają tylko kod i bezpieczne przykłady.

Sprawdź wynik: Repozytorium zawiera kod i bezpieczny przykład; pliki danych i sekretów są ignorowane przez Git.

2. Utwórz nowy projekt/usługę Railway

W Railway wybierz New Project → Deploy from GitHub repo lub odpowiadającą temu opcję i wskaż repozytorium bota. Zezwól GitHubowi na dostęp wyłącznie do potrzebnego repo. Jeśli package.json i Dockerfile są od razu w głównym katalogu repo, zostaw domyślny Root Directory. Jeżeli przesłałeś cały nadrzędny folder i pliki są w bot-starter/, ustaw Root Directory na bot-starter. Dostarczony Dockerfile wybiera Node.js 24.19+. Nie zmieniaj katalogu wdrażania ani domen działającej strony dawkoz.com.

Sprawdź wynik: Nowa usługa ma odrębną nazwę i wdraża właściwy kod, a dawkoz.com nadal działa.

3. Dodaj trwały dysk bazy danych

W projekcie Railway dodaj Volume i ustaw mount path /data. W Variables ustaw DATA_DIR=/data. Baza SQLite musi znajdować się na tym volume, nie na tymczasowym dysku kontenera. Sprawdź koszty volume i kopii. Uruchom tylko jedną replikę tej instancji; ten wariant nie jest klastrem kilku procesów piszących do wspólnej bazy. Własną kopię bazy wykonuj spójnym sposobem zgodnym ze starterem/SQLite, a nie przypadkowym kopiowaniem jednego pliku podczas zapisu WAL.

Sprawdź wynik: Plik SQLite znajduje się na volume, a próba kontrolowanego restartu zachowuje fikcyjne zgłoszenie i licznik.

4. Wpisz zmienne w zakładce Variables

W Variables dodawaj wartości według tabeli poniżej. W New Variable utwórz CLIENT_CONFIG_JSON i do pola wartości wklej całą treść zatwierdzonego config/client.json. To zawartość JSON, nie nazwa pliku i nie ścieżka z Twojego komputera. Nie dokładaj samodzielnie dodatkowych cudzysłowów wokół całego dokumentu. Korzystając z edytora wszystkich zmiennych naraz, pamiętaj, że wtedy JSON musi być wartością tekstową właściwie zakodowaną; dla początkującego prostsze jest oddzielne New Variable. Początkowo ALLOW_LIVE_CALLS pozostaje false. Uwaga: BOT_MODE=live przy ALLOW_LIVE_CALLS=false celowo zatrzymuje proces; to stan przygotowania, nie działająca usługa. Po uzupełnieniu wszystkich zmiennych i adresu ustaw true wyłącznie do kontrolowanego testu z LIVE_TEST_ONLY=true. Demo wiąże serwer z localhost, więc nie wykorzystuj go do publicznego hostingu.

Sprawdź wynik: Usługa uruchamia się bez drukowania sekretów; nie przyjmuje jeszcze przypadkowego ruchu firmy.

5. Wygeneruj adres HTTPS

W Settings → Networking / Public Networking użyj Generate Domain lub odpowiednika w Twoim panelu. Skopiuj dokładny adres https://…up.railway.app. Wpisz go do PUBLIC_BASE_URL bez końcowego ukośnika, ścieżki, parametrów i #. Ponownie zastosuj wdrożenie. Do podpisu Twilio potrzebny jest ten sam zewnętrzny adres; adres localhost i adres prywatny Railway nie działają jako publiczny webhook. Nie twórz DNS metodą kopiowania losowego hosta z instrukcji.

Sprawdź wynik: Publiczny adres działa po HTTPS, certyfikat jest poprawny, a konfiguracja używa dokładnie tego samego adresu.

6. Sprawdź start, zdrowie i trwałość

Sprawdź log wdrożenia i otwórz https://TWÓJ-HOST/healthz. Nie dodawaj własnego PORT do Railway, jeśli platforma już go przekazuje; lokalnie domyślnie używany jest 3001. Sprawdź konfigurację poleceniem npm run check na tym samym zestawie ustawień. Npm run status pokazuje zbiorczy stan limitu i kolejki, bez odczytywania danych klienta w publicznym /healthz. Następnie kontrolowany restart na fikcyjnych danych ma zachować bazę. Sam status zdrowia nie sprawdza wszystkich dostawców.

Sprawdź wynik: Proces działa w jednej replice, baza pozostała po restarcie, a test konfiguracji nie zgłasza braków.

E. Połącz Twilio z tym botem

Adres / metoda	Gdzie go używasz
GET /healthz	Przeglądarka i healthcheck hostingu. Tylko stan zdrowia, nie dane klientów.
GET /demo	Lokalny tryb demo. To nie jest adres dla połączeń i nie powinien być demonstratorem produkcji z danymi klienta.
POST /voice/incoming	Twilio → A call comes in → Webhook → HTTP POST.
POST /voice/turn?nonce=…	Program tworzy go sam. Nie wpisuj ręcznie w konfiguracji numeru i nie usuwaj nonce.
POST /voice/status	Twilio → status callback rozmowy. Potrzebny do uzgodnionego rozliczenia i zakończenia stanu połączenia.

Przykład składania adresu: jeśli PUBLIC_BASE_URL to https://firma-demo.up.railway.app, początek rozmowy ma adres https://firma-demo.up.railway.app/voice/incoming. Host firma-demo jest tylko ilustracją; przepisz host przydzielony Twojej usłudze. Nigdy nie wpisuj w tych polach domeny strony dawkoz.com tylko dlatego, że tam jest instrukcja.

Adresy webhooków i pierwsza próba telefoniczna

1. Wybierz dokładnie właściwy numer w Twilio

W Console → Phone Numbers → Manage → Active numbers otwórz numer testowy tej firmy. Sprawdź numer i konto jeszcze raz. Zapisz poprzednie ustawienia Voice przed zmianą. Nie edytuj numeru innej firmy ani głównego numeru działającej strony.

Sprawdź wynik: SID konta i numer zgadzają się z konfiguracją instancji bota.

2. Ustaw przychodzące połączenie jako webhook POST

W Voice Configuration numeru odszukaj A call comes in. Wybierz Webhook, wklej https://TWÓJ-HOST/voice/incoming i ustaw HTTP POST. TWÓJ-HOST zamień na rzeczywisty host z PUBLIC_BASE_URL. Zapisz zmianę. Nie wklejaj /demo, localhost ani /voice/turn. Adres turn z bezpiecznym nonce generuje sam program i nie jest ręcznie konfigurowanym początkiem połączenia.

Sprawdź wynik: Pole zawiera publiczny HTTPS oraz prawidłową ścieżkę przychodzącego połączenia; metoda to POST.

3. Ustaw callback statusu połączenia

W części statusów rozmowy, np. Call status changes lub ustawieniach zaawansowanych, wpisz https://TWÓJ-HOST/voice/status i POST. Jeśli obecny panel numeru nie pokazuje tego pola, użyj dokumentacji konfiguracji IncomingPhoneNumber i jego statusCallback albo poproś ChatGPT o ustawienie dokładnie tej właściwości przez API przy użyciu sekretów z konta, bez wklejania ich do czatu. Nie podstawiaj callbacku SMS. Test zakończonej rozmowy musi potwierdzić, że ten endpoint dostaje prawdziwy status z właściwego konta.

Sprawdź wynik: Testowa zakończona rozmowa wywołuje prawdziwy callback i pozwala rozliczyć czas; nie jest liczona wyłącznie z zegara przeglądarki.

4. Sprawdź ochronę webhooków

Adres webhooka jest publiczny, ale żądanie musi mieć poprawny podpis Twilio i pasujące konto oraz numer docelowy. Ręczne otwarcie endpointu w przeglądarce nie jest prawidłowym testem połączenia. Nie wyłączaj weryfikacji podpisów, żeby usunąć błąd 403.

Sprawdź wynik: Niepodpisane żądanie jest odrzucane, a rzeczywista próba z właściwego konta przechodzi.

5. Włącz wyłącznie uzgodniony tryb próbny live

Po sprawdzeniu zmiennych ustaw BOT_MODE=live, ALLOW_LIVE_CALLS=true i pozostaw LIVE_TEST_ONLY=true. W ALLOWED_CALLERS wpisz wyłącznie własny dozwolony numer próbny w formacie +48…; przy kilku numerach zachowaj format opisany w README. Nie wpisuj tam numeru bota zamiast dzwoniącego. Ustaw bieżący jawny okres BILLING_PERIOD_START / BILLING_PERIOD_END i zrób małą płatną próbę. Ta zmiana nie jest zgodą na przełączenie całego ruchu klienta.

Sprawdź wynik: Tryb live jest świadomym ustawieniem, a nie skutkiem wpisania pierwszego klucza API.

6. Zadzwoń i przejdź jedną całą sprawę

Z dozwolonego własnego numeru zadzwoń na numer Twilio. Posłuchaj powitania, odpowiedz na trzy pytania, popraw jedną informację i potwierdź podsumowanie. Poczekaj na zakończenie rozmowy i dostarczenie sprawy. Zapisz CallSid oraz koszt w panelach dostawców bez przesyłania do czatu danych rozmówcy.

Sprawdź wynik: Zgłoszenie pojawia się u właściwego odbiorcy, a rozliczenie obejmuje rzeczywisty czas po zaokrągleniu i ewentualne płatne odcinki.

7. Sprawdź głos i język na prawdziwej rozmowie

Polski język w konfiguracji nie gwarantuje identycznej dostępności każdego głosu i modelu rozpoznawania mowy na koncie. Jeśli provider odrzuca głos, wybierz jawnie obsługiwany polski głos według jego dokumentacji i ponów próbę. Nie zmieniaj automatycznie na obcy język ani droższy wariant bez sprawdzenia.

Sprawdź wynik: Słowa, numery i typowy adres są zrozumiałe; przerwy między turami odpowiadają temu, co klient zaakceptował.

F. Sprawdź ograniczenia, zanim nazwiesz usługę gotową

Próba	Co robisz	Wynik wymagany
Brak danych	Milcz lub powiedz „nie wiem” zamiast wymaganego pola.	Bot dopytuje w granicach prób; nie wymyśla danych i kończy kontrolowanie.
FAQ	Zadaj zatwierdzone pytanie w środku zbierania danych.	Pada zatwierdzona odpowiedź i powrót do właściwego pytania.
Nieznany temat	Zapytaj o nieuzgodnioną cenę lub termin.	Bot nie obiecuje wyniku i oddaje sprawę do ustalonej weryfikacji.
Potwierdzenie	Po odczytaniu podsumowania powiedz „nie”, a potem popraw dane.	Nieprzyjęta wersja nie trafia jako potwierdzona sprawa.
Błąd OpenAI	Użyj niewłaściwego klucza w środowisku testowym.	Brak niekontrolowanej rozmowy, wycieku sekretu i automatycznego przejścia na inny rachunek.
Błąd Telegram	Tymczasowo wskaż niepoprawny identyfikator testowego czatu.	Sprawa pozostaje w bazie, a status nie twierdzi, że została doręczona.
Ponowione zdarzenie	Odtwórz ten sam callback w lokalnych testach.	Jedno logiczne zdarzenie i brak wielokrotnego naliczenia.

Próba	Co robisz	Wynik wymagany
Limit	Zmniejsz limit testowy i rozpocznij kolejną rozmowę.	Nowe płatne działanie jest wstrzymane; nie włącza się płatny fallback.
Dwie rozmowy	W dozwolonym teście spróbuj dwóch równoczesnych połączeń.	Jednoreplikowy wariant kontroluje równoczesność i nie przekracza rezerwacji limitu.
Maksymalny czas	Przeprowadź kontrolowaną dłuższą próbę.	Działa ustalone kończenie i limit; dopiero realny test potwierdza zachowanie u dostawcy.
Restart	Zrestartuj usługę z fikcyjną sprawą w bazie.	Sprawa i naliczenie nie znikają; nie jest wysyłany ponownie komplet dawnych wiadomości.
Inna firma	W teście użyj innego konta lub numeru docelowego.	Żądanie nie trafia do konfiguracji i odbiorcy tej firmy.

Dlaczego przy pozostałych 1–4 minutach nowa rozmowa może być odrzucona
Pilotażowy starter rezerwuje pełne pięć minut na każdą przyjmowaną rozmowę, przy jednej równoczesnej rozmowie. Gdy wolnych jest mniej niż pięć, nie przyjmie następnej. To konserwatywna reguła testowego wariantu, a nie pełne odwzorowanie każdej zasady końcówki pakietu z oferty. Po zakończeniu rzeczywistej rozmowy rozliczenie wymaga wiarygodnego callbacku. Brak callbacku nie może automatycznie uwolnić rezerwacji na podstawie domysłu.

Maksymalny czas i odrzucenie też trzeba sprawdzić u operatora
Starter próbuje ustawić limit połączenia Twilio i kończy proces zgodnie z czasem aplikacji. Samo wysłanie timeLimit=300 lub odpowiedź HTTP 200 nie dowodzi jeszcze, jak rzeczywiście rozliczy się konkretne przychodzące połączenie na koncie. Przeprowadź kontrolowaną próbę. Odrzucenie połączenia w Twilio nie musi oznaczać zerowego kosztu u operatora, który przekierował do niego rozmowę.

Ograniczenie budżetu wymaga pomiaru
Rezerwacja minut, limit czasu i alert dostawcy nie są matematyczną gwarancją pełnego kosztu faktury: dochodzą stawki głosu/rozpoznawania mowy, modelu, numeru, infrastruktury i opóźnienia callbacków. Próg kosztowy z oferty nie jest znalezioną taryfą dostawcy. Nie przyjmuj klienta w danej cenie, dopóki nie zmierzysz rachunku dla docelowego stosu i nie potwierdzisz zatrzymania przy limicie.

Gotowość do przeniesienia rzeczywistego ruchu

☐ Przeprowadzono co najmniej 20 zróżnicowanych rozmów i zapisano wyniki, czas oraz koszty.

☐ Klient zatwierdził powitanie, pytania, FAQ, podsumowanie i zachowanie przy braku danych.

☐ Przetestowano stan bazy po restarcie, poprawne naliczenie, limit, brak nieuzgodnionego fallbacku i awarię dostawcy.

☐ Ustalono sprzedawcę, administratora, dostawców, informowanie o AI i ewentualne przetwarzanie danych; brakujące kwestie nie zostały zastąpione deklaracją „RODO jest”.

☐ Istnieje kopia konfiguracji i bazy oraz sprawdzony sposób powrotu do poprzedniej obsługi telefonu.

☐ Zapisano datę odbioru i uruchomienia, osobę akceptującą, numer oraz pierwszy okres rozliczeniowy.

☐ Brakujące elementy pełnej oferty — w szczególności SMS — zostały wdrożone lub wyraźnie wyłączone w indywidualnie uzgodnionym zakresie.

G. Pierwsze dni po uruchomieniu

```
npm run check
npm run status
npm run jobs
```

Uruchamiaj narzędzia w tej samej instancji i z tą samą konfiguracją/dyskiem, której stan chcesz sprawdzić. Lokalny npm run status nie odczytuje automatycznie bazy Railway. Nie pobieraj bazy klienta do dowolnego katalogu tylko po to, żeby zobaczyć licznik. Narzędzie jobs nie stanowi zgody na ponowne wysłanie wiadomości z niepewnym doręczeniem.

Codzienna kontrola operatora

1. Sprawdź, czy nowa sprawa została przyjęta i doręczona

Npm run status pokazuje zużycie i zbiorcze stany. Npm run jobs wypisuje identyfikatory i statusy, bez treści zgłoszeń; nie uruchamia wysyłki. Serwer sam obsługuje pending/retry. Delivered oznacza potwierdzony identyfikator Telegram, failed wymaga naprawy konfiguracji, unknown oznacza niepewność po timeout/błędzie. Najpierw ręcznie sprawdź prywatny czat po ID. Jeżeli wiadomość tam jest, użyj npm run jobs -- --delivered ID. Dopiero gdy potwierdzisz jej brak i poprawisz przyczynę, użyj npm run jobs -- --retry ID. Ręczne ponowienie może stworzyć duplikat przy błędnej ocenie. Wykonuj to w tej samej instancji i na jej dysku, nie w lokalnej kopii demonstracyjnej.

Sprawdź wynik: Każde przyjęte zgłoszenie ma widoczny stan; sprawy niedoręczone nie zniknęły.

2. Porównaj lokalny licznik z dostawcą

Sprawdź ostatnie zakończone połączenia w Twilio i zużycie projektu OpenAI. Zwróć uwagę na zaokrąglenia i opóźnienie statusów. Przy dużej rozbieżności zatrzymaj nowe próby oraz zdiagnozuj przyczynę, zanim uznasz pakiet za bezpieczny.

Sprawdź wynik: Wiesz, skąd bierze się rachunek i czy założenia ceny nadal są prawdziwe.

3. Wprowadzaj małe, wersjonowane zmiany

Najpierw zmień konfigurację na teście, wykonaj próbę na danych fikcyjnych i zachowaj poprzednią wersję. Aktualizacja FAQ nie może przy okazji podmienić odbiorcy innej firmy. Zapisz datę i zakres zmiany oraz czas opieki.

Sprawdź wynik: Po zmianie nadal działa cały przebieg, a klient wie, co zostało zmienione.

4. Kończ współpracę świadomie

Przed wyłączeniem usługi ustal numer, eksport danych, retencję i płatności zgodnie z umową. Wyłącz źródło ruchu i zaplanowane procesy, cofnij niepotrzebne uprawnienia, a potem usuń dane w uzgodnionym trybie. Nie kasuj projektu i bazy na prośbę nieuprawnionej osoby.

Sprawdź wynik: Nie działa osierocony płatny bot; firma ma potwierdzenie przekazania i wie, co nadal opłaca.

Brak callbacku, retencja i wznowienie okresu

1. Wyjaśnij brak końcowego statusu

Jeśli activeOrAwaitingCallback pozostaje dodatnie po zakończeniu rozmowy, nie zeruj bazy ani nie zwalniaj limitu według zegara. W Twilio sprawdź CallSid i końcowy status. W terminalu tej samej usługi uruchom npm run reconcile -- CA... z rzeczywistym CallSid. Narzędzie odczytuje rekord przez uwierzytelnione API Twilio, sprawdza konto, numer, status i czas, a dopiero potem rozlicza rezerwację. Gdy rekord nie jest zakończony lub niepasujący, pozostawia blokadę. Odczyt statusu wymaga sieci; nie jest częścią offline.

Sprawdź wynik: Rezerwacja jest rozliczona jeden raz według rekordu dostawcy; żaden trwający telefon nie zniknął z limitu.

2. Ustal i wykonuj retencję

Starter czyści treść zgłoszenia, odpowiedzi i zapisane XML po contentRetentionDays, domyślnie7 dniach, przy starcie i co godzinę. Ręcznie: npm run cleanup. Minimalne metadane zakończonych rozmów, ID i statusy pozostają do400 dni; to domyślna granica prototypu do uzasadnienia i ewentualnego skrócenia z administratorem przed produkcją. Nierozliczone połączenia pozostają do wyjaśnienia. Osobno ustal usuwanie z Telegram, kopii, Twilio i OpenAI — ta komenda ich nie usuwa.

Sprawdź wynik: Klient zaakceptował konkretne okresy i wie, gdzie dane pozostają; w teście po czyszczeniu nie ma treści w SQLite.

3. Odnów okres rozliczeniowy świadomie

Przed końcem okresu sprawdź umowę, płatność i nierozliczone połączenia. Ustaw następny uzgodniony BILLING_PERIOD_START oraz BILLING_PERIOD_END z pełną strefą, zachowując bazę i historię. Nie zeruj licznika przez usuwanie danych. Okres po zakończeniu blokuje nowe rozmowy; prototyp nie ma automatycznego rozliczania ani alertów80/100%. Wymaga kontroli operatora i uzupełnienia tych funkcji dla pełnej oferty.

Sprawdź wynik: Nowy okres odpowiada aktywacji i podpisanym warunkom; stare zużycie pozostało w historii.

Node.js — oficjalne pobranie
https://nodejs.org/en/download

Twilio — Voice i TwiML Gather
https://www.twilio.com/docs/voice/twiml/gather

Twilio — podpisy żądań
https://www.twilio.com/docs/usage/security

OpenAI — Responses API
https://platform.openai.com/docs/api-reference/responses

Telegram — Bot API
https://core.telegram.org/bots/api

Railway — volumes
https://docs.railway.com/guides/volumes

2. Kolejna firma — osobny bot od briefu do odbioru

Nie dopisuj nowego klienta do działającej bazy pierwszej firmy. Przygotuj odrębną instancję i przejdź poniższą listę.

Osobna konfiguracja to podstawa izolacji
Starter obsługuje jedną firmę. Kolejna firma oznacza odrębną usługę, dysk, numer i konto/projekt dostawców lub wyraźnie odizolowany zasób, klucze oraz odbiorcę. Panel dokumentów DAWKOZ nie jest systemem wielu kont klientów. Nie udostępniaj klientowi swojego hasła administracyjnego.

1. Ustal co firma zamawia

1. Wybierz jeden proces

W rozmowie z klientem zapisz cel, np. przyjęcie usterek i przekazanie ich jednej osobie. Wypisz pytania wymagane, zatwierdzone FAQ, godziny pracy, sposób kontaktu z człowiekiem oraz sytuacje awaryjne. Ten starter nie przekierowuje do pracownika, nie rezerwuje terminu, nie potwierdza ceny i nie wysyła SMS. Jeśli firma tego oczekuje, zapisz dodatkowe prace przed aktywacją.

Sprawdź wynik: Klient potwierdził cel i ograniczenia. Nie obiecujesz funkcji, których program nie realizuje.

2. Przygotuj dokumenty

W panelu DAWKOZ wybierz ofertę i warunki pakietu. Sprawdź tożsamość sprzedawcy, cenę, VAT, integrację, limit, materiały, odbiór, opiekę i role danych. Starter nie spełnia sam całego zakresu Bot AI, np. SMS i alertów — uzupełnij kod lub uzgodnij osobny zakres pilotażu. Umowę, wymagane powierzenie i zaliczkę załatw zgodnie ze wspólnym procesem podręcznika.

Sprawdź wynik: Zakres podpisany odpowiada rzeczywiście zaplanowanej realizacji i kosztom.

3. Zbierz zatwierdzone materiały

Poproś o poprawną nazwę firmy, powitanie z informacją o AI, od1 do6 pytań formularza, do20 FAQ oraz docelowego odbiorcę. Dla każdego pola określ dozwoloną długość i sposób sprawdzenia. Uzgodnij minimalny zakres danych, informację dla dzwoniącego i retencję. Nie używaj kontaktów ani scenariusza innego klienta jako danych nowej firmy.

Sprawdź wynik: Masz wersję materiałów z datą i potwierdzeniem klienta.

2. Przygotuj nową instancję

1. Rozpakuj świeży ZIP

Pobierz kod z panelu i rozpakuj do nowego folderu, np. bot-firma-b-test. Nie kopiuj całego katalogu działającej firmy, bo zawiera jej .env i bazę. Uruchom npm run init oraz npm test. Nie przenoś plików SQLite, kluczy ani prawdziwych zgłoszeń.

Sprawdź wynik: Nowy folder nie zawiera danych, tokenów i odbiorcy poprzedniej firmy.

2. Ustaw nową tożsamość w JSON

W config/client.json ustaw nowy unikalny clientId, np. firma-b, oraz zatwierdzone companyName. Zmień powitanie, processDescription, pola i FAQ. Zachowaj ograniczenia zgodne z umową. Pierwszy start wiąże bazę z clientId i companyName; przy komunikacie o innej firmie sprawdź DATA_DIR i volume zamiast usuwać zabezpieczenie. Zmiana samej nazwy w istniejącej bazie też wymaga świadomej migracji, której starter nie automatyzuje.

Sprawdź wynik: Npm run check przechodzi, a offline demo mówi wyłącznie o nowej firmie.

3. Przećwicz proces bez API

Uruchom npm start, otwórz localhost:3001/demo i przejdź pytania na danych fikcyjnych, potwierdź i odrzuć podsumowanie. Automatyczne npm run demo sprawdza dołączony przykładowy scenariusz, więc nie zastępuje ręcznego testu zmienionych pytań klienta. Nie traktuj tekstowej próby jako testu jakości głosu.

Sprawdź wynik: Kolejność i treść pytań są poprawne, a brak potwierdzenia nie tworzy sprawy.

4. Utwórz osobną usługę Railway i volume

Wykonaj sekcję D poprzedniego rozdziału na odrębnej usłudze, z jedną repliką i pustym trwałym /data. Zatwierdzony JSON wklej do CLIENT_CONFIG_JSON. Ustaw nowe klucze, odbiorcę i numer; PUBLIC_BASE_URL musi wskazywać dokładnie nowy host. Obejrzyj nazwy projektu i zmiennych przed zapisaniem. Nie zmieniaj volume ani numeru poprzedniego klienta.

Sprawdź wynik: Dwie firmy mają różne usługi, numery i bazy; restart nowej nie wpływa na starą.

5. Podłącz konta i numer

Wykonaj sekcje C i E poprzedniego rozdziału. Potwierdź uprawnienia klienta i koszty u dostawców. Włącz live wyłącznie z listą własnych numerów próbnych. W konfiguracji nowego numeru ustaw /voice/incoming oraz /voice/status z nowego hosta. Sekretów nie wklejaj do czatu, PDF, zdjęcia ani repozytorium.

Sprawdź wynik: Testowy telefon trafia wyłącznie do nowej instancji, a zgłoszenie do właściwego odbiorcy.

Test izolacji	Jak sprawdzić	Oczekiwany wynik
Nazwa i pytania	Zadzwoń na oba numery testowe.	Każdy bot przedstawia właściwą firmę i jej pytania.
Odbiorca	Przyjmij fikcyjną sprawę dla firmyB.	Wiadomość trafia do odbiorcyB, bez kopii doA.
Konto i numer	Uruchom test podpisów i konfiguracji.	Niewłaściwe konto lub To jest odrzucone.
Zużycie	Porównaj npm run status w każdej instancji przed i po próbie.	Zmienia się tylko właściwy licznik; callback rozlicza właściwe CallSid.
Restart	Kontrolowanie zrestartuj nową usługę.	Zgłoszenie i licznik pozostają, druga usługa działa dalej.
Brak callbacku	Sprawdź procedurę reconcile na zakończonym rekordzie testowym.	Rezerwacja nie znika na sam timeout i rozlicza się jeden raz.

3. Odbierz usługę i przekaż obsługę

1. Wykonaj próby i policz koszt

Zapisz co najmniej20 rozmów według tabeli testów. Sprawdź krótkie i długie odpowiedzi, ciszę, hałas, odmowę podsumowania, nieznane pytania, awarie dostawców, limit minut, maksymalny czas i właściwe naliczenie. Porównaj rachunki i opóźnienie z ofertą. Nie uruchamiaj abonamentu tylko dlatego, że /healthz zwraca200.

Sprawdź wynik: Wyniki i koszty zapisano, klient sprawdził pełny przebieg.

2. Zamknij braki zakresu

Jeżeli podpisana usługa obejmuje SMS, alerty, przekierowanie, inne narzędzie albo panel, wykonaj i przetestuj te elementy przed odbiorem. Nie zastępuj brakujących zabezpieczeń samym zapisem w umowie. Dla pilotażu nazwij ograniczenia wprost i ustal termin oraz kryteria rozszerzenia.

Sprawdź wynik: Każda pozycja zamówienia ma wynik testu lub uzgodnione wyłączenie w nowym zakresie.

3. Zatwierdź start i plan awaryjny

Zbierz uwagi oraz wyraźny protokół odbioru, rozlicz drugie50% i ustal datę aktywacji zgodnie z umową. Zapisz stare ustawienia numeru oraz sposób cofnięcia przekierowania. Dopiero w uzgodnionym oknie zdejmij LIVE_TEST_ONLY. Przekaż firmie miejsce odbioru spraw, granice procesu i kontakt wsparcia.

Sprawdź wynik: Firma wie, jak znaleźć zgłoszenie i jak obsłużyć klienta przy awarii bota.

4. Przez pierwsze dni kontroluj codziennie

Porównuj zgłoszenia z kolejką, statusy końcowe rozmów, zużycie minut, koszt i jakość. Niepewne doręczenia sprawdzaj ręcznie przed ponowieniem. Kontroluj retencję i kopię/odtworzenie. Przed końcem okresu odnowienie dat i płatność wymagają kontroli; starter nie wykonuje ich automatycznie.

Sprawdź wynik: Nie ma niewyjaśnionych spraw, osieroconych rezerwacji ani nieuzgodnionych kosztów.

```
KARTA WDROŻENIA (bez sekretów)
Projekt i clientId: [...]
Nazwa firmy / osoba uprawniona: [...]
Cel i wyłączenia: [...]
Wersja pytań i FAQ / akceptacja: [...]
Usługa Railway / host / volume: [...]
Numer i konto dostawcy (bez tokenu): [...]
Odbiorca Telegram / test potwierdzony: [...]
Daty okresu i limit: [...]
Retencja / backup / test odtworzenia: [...]
Wyniki20 prób / pomiar kosztu: [...]
Braki względem oferty i ich zamknięcie: [...]
Protokół odbioru / data startu: [...]
Powrót do poprzedniej obsługi / kontakt: [...]
```

Telefon a aplikacja klienta
Bot działa jako osobna usługa serwerowa. Przyszły portal/aplikacja wymaga osobnego uwierzytelniania, autoryzacji do konkretnej firmy i zakresu danych. Nie publikuj pliku SQLite, /demo z danymi rzeczywistymi ani sekretów jako sposobu udostępnienia zgłoszeń. Obecny panel DAWKOZ służy operatorowi.